

Intermediate Code Generation using Syntax-Directed Definitions (SDD) and Syntax-Directed Translation (SDT)

Lecture Notes based on *Compilers: Principles, Techniques, and Tools* (the “Dragon Book”),
2nd Edition, Aho, Lam, Sethi, Ullman – Chapters 5 and 6

Lecture Plan (suggested: 2 lectures $\times$ 75 min)

Lecture 1 Role of intermediate code; forms of IR (syntax tree, DAG, three-address code, quadruples, triples); recap of SDD and SDT (synthesized/inherited attributes, S- and L-attributed definitions, translation schemes); translation of expressions, assignments, and array references; type conversion.

Lecture 2 Boolean expressions and short-circuit code; flow-of-control statements with inherited labels; backpatching; switch statements and procedure calls (briefly); worked examples, exercises.

Contents

1	Introduction: Why Intermediate Code?	2
1.1	Advantages of using an intermediate representation	2
2	Variants of Syntax Trees and Intermediate Representations	2
2.1	Syntax trees (abstract syntax trees)	2
2.2	Directed acyclic graphs (DAGs) for expressions	2
2.2.1	The value-number method for constructing DAGs	3
2.3	Three-address code (TAC)	3
2.3.1	Addresses and instructions	4
2.3.2	Representations of three-address code	4
2.3.3	Static single-assignment (SSA) form – a brief note	5
3	Syntax-Directed Definitions (SDD): A Recap	5
3.1	Synthesized and inherited attributes	6
3.2	Annotated parse trees and dependency graphs	7
3.3	Classes of SDDs that are easy to implement	7
3.4	SDD for constructing syntax trees	8

4	Syntax-Directed Translation Schemes (SDT)	8
4.1	Postfix translation schemes	9
4.2	SDTs with actions inside productions	9
4.3	Eliminating left recursion from SDTs and marker nonterminals	10
4.4	Converting an L-attributed SDD into an SDT	10
5	Translation of Expressions and Assignments	10
5.1	SDD for three-address code	10
5.2	Incremental translation (SDT with gen emitting directly)	12
5.3	Addressing array elements	12
5.4	Translation of array references	13
6	Type Checking and Type Conversion	13
6.1	Type conversion (coercion)	14
7	Control Flow: Boolean Expressions and Statements	14
7.1	Short-circuit (jumping) code	14
7.2	Flow-of-control statements: SDD with inherited labels	15
7.3	SDD for boolean expressions (jumping code)	17
7.4	Avoiding redundant gotos (fall-through)	17
8	Backpatching	18
8.1	The problem and the idea	18
8.2	Boolean expressions	18
8.3	Flow-of-control statements	19
8.4	Comparison: inherited labels vs. backpatching	21
9	Switch Statements and Procedure Calls (Brief)	21
9.1	Switch statement	21
9.2	Procedure calls	21
10	A Complete Worked Example	22
11	Summary and Key Takeaways	23
12	Practice Problems	23

1 Introduction: Why Intermediate Code?

In the *analysis-synthesis* model of a compiler, the **front end** analyses the source program and produces an **intermediate representation (IR)**; the **back end** synthesises the target program from that IR.

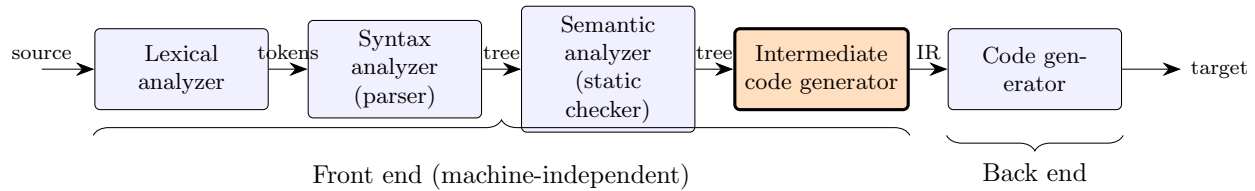


Figure 1: Position of the intermediate code generator in a compiler. The shaded (thick) box is the topic of these notes. In practice, static checking and intermediate code generation are often done together in one pass.

1.1 Advantages of using an intermediate representation

- **Retargeting.** A compiler for m languages and n machines needs only m front ends and n back ends (instead of $m \times n$ compilers) if they share one IR.
- **Machine-independent optimisation.** Optimisations (common sub-expression elimination, constant folding, loop-invariant code motion, ...) are applied once on the IR.
- **Simplicity.** Complex source constructs are lowered to a few simple operations, making code generation easier.

Static checking vs. intermediate code

Static checking includes type checking and syntactic checks that the grammar cannot enforce (e.g., **break** must be inside a loop). Dragon Book Chapter 6 treats type checking and intermediate-code generation together, because both are done by attaching attributes and actions to the grammar.

2 Variants of Syntax Trees and Intermediate Representations

2.1 Syntax trees (abstract syntax trees)

A **syntax tree** is a condensed form of the parse tree: operators and keywords appear as *interior nodes*, and operands as children. Chains of single productions disappear.

2.2 Directed acyclic graphs (DAGs) for expressions

A **DAG** identifies the *common sub-expressions*: a node is shared if the same sub-expression appears more than once. It shows the same information as a syntax tree but is more compact, and it directly suggests how to generate efficient code that computes a common sub-expression only once.

Example: tree vs. DAG for $a + a * (b - c) + (b - c) * d$

Figure 2 shows the syntax tree (left) and the DAG (right). In the DAG, the leaf a and the sub-expression $b - c$ are each shared by two parents.

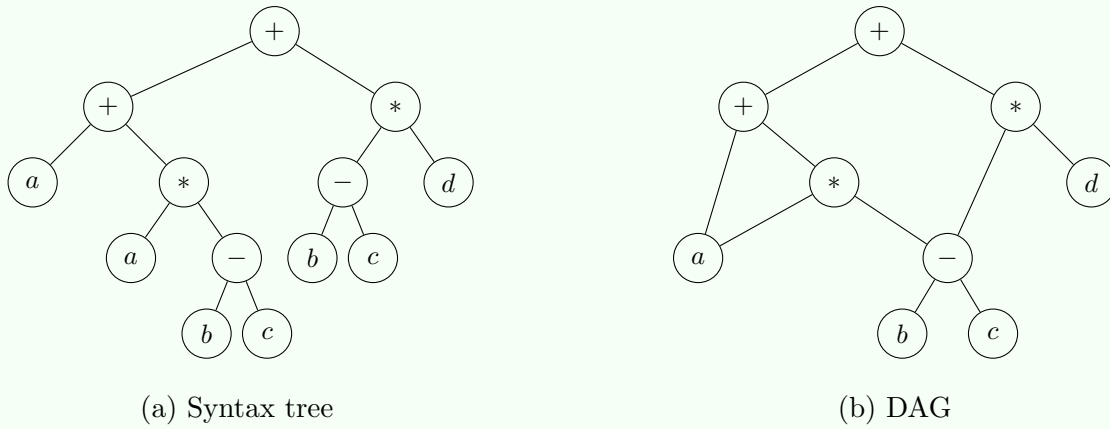


Figure 2: Syntax tree and DAG for $a + a * (b - c) + (b - c) * d$.

2.2.1 The value-number method for constructing DAGs

Nodes are stored in an array of records (op, left, right); the index of a record is its **value number**. Before creating a node $\langle op, l, r \rangle$, search the array; if an identical node exists, return its number, otherwise create a new one (use a hash table for speed).

Value no.	Op	Left	Right
1	id a	—	—
2	id b	—	—
3	id c	—	—
4	—	2	3
5	*	1	4
6	+	1	5
7	id d	—	—
8	*	4	7
9	+	6	8

When a is seen the second time, and when $b - c$ is seen the second time, the existing numbers 1 and 4 are returned – no new nodes are made.

2.3 Three-address code (TAC)

In **three-address code** every instruction has *at most one operator on the right side* and at most three addresses (two operands and one result). Compound expressions are broken into temporaries, so the order of evaluation is made explicit.

For $a + a * (b - c) + (b - c) * d$ the TAC (from the DAG, reusing $t_1 = b - c$) is:

`t1 = b - c`

```

t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4

```

Without the DAG (from the tree) $b - c$ would be computed twice.

2.3.1 Addresses and instructions

An **address** can be: (i) a *name* (source variable; a pointer to its symbol-table entry), (ii) a *constant*, or (iii) a compiler-generated *temporary*.

Instruction form	Meaning
$x = y \text{ op } z$	binary assignment
$x = \text{op } y$	unary assignment (minus , not , conversions)
$x = y$	copy
goto L	unconditional jump
if x goto L / ifFalse x goto L	conditional jump on truth value
if x relop y goto L	conditional jump on comparison
param x, call p, n, y = call p, n, return y	procedure calls
$x = y[i], x[i] = y$	indexed copy (memory at address $y + i$)
$x = \&y, x = *y, *x = y$	address and pointer assignments
L:	label (a symbolic name for an instruction)

Table 1: Common three-address instruction forms.

2.3.2 Representations of three-address code

Take the source statement $a = b * -c + b * -c$.

(a) Three-address code

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

```

(b) Quadruples – (op, arg1, arg2, result)

	op	arg ₁	arg ₂	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

(c) **Triples** – (op, arg1, arg2); a result is referred to by the *position* of its triple.

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

(d) **Indirect triples** – a list of pointers to triples.

instruction	pointer	
35	(0)	
36	(1)	
37	(2)	(triples as in (c))
38	(3)	
39	(4)	
40	(5)	

	Advantage	Disadvantage
Quadruples	Results are named explicitly (temporaries); instructions can be moved freely during optimisation.	Needs extra space for temporaries and symbol-table entries.
Triples	No temporaries; more compact.	Moving a triple changes positions, so all references to it must be updated.
Indirect triples	Reordering only rearranges the pointer list, so triples keep their positions.	Extra level of indirection; an additional list is needed.

Table 2: Comparison of the three representations.

2.3.3 Static single-assignment (SSA) form – a brief note

In **SSA form** every variable is assigned exactly once; different assignments to the same source variable get different subscripts, and a ϕ -function merges values at control-flow joins:

p = a + b	p1 = a + b
q = p - c	q1 = p1 - c
p = q * d	p2 = q1 * d
p = e - p	p3 = e - p2
q = p + q	q2 = p3 + q1

```
if (flag) x = -1; else x = 1; y = x * a;
==>  if (flag) x1 = -1; else x2 = 1; x3 = phi(x1, x2); y = x3 * a;
```

3 Syntax-Directed Definitions (SDD): A Recap

Syntax-Directed Definition (SDD)

A **syntax-directed definition** is a context-free grammar together with **attributes** attached to grammar symbols and **semantic rules** attached to productions. For a production $A \rightarrow \alpha$ every rule has the form

$$b = f(c_1, c_2, \dots, c_k)$$

where f is a function and either

- (1) b is a **synthesized** attribute of A and $c_1, \dots, c_k$ are attributes of the symbols in α (the children) or of A itself; or

- (2) b is an **inherited** attribute of a symbol B in α and $c_1, \dots, c_k$ are attributes of A (the parent) or of symbols in α (the siblings), or of B itself.

3.1 Synthesized and inherited attributes

- A **synthesized attribute** at a parse-tree node N is computed from attributes of N 's *children* (and N itself). Information flows *bottom-up*. Terminals have synthesized attributes only (supplied by the lexer, e.g., **digit.lexval**).
- An **inherited attribute** at N is computed from attributes of N 's *parent and siblings*. Information flows *top-down / sideways*. Useful when the structure of the parse tree does not match the abstract syntax, e.g., passing a type to a list of identifiers, or passing the label to jump to.

Example: synthesized attributes only (desk calculator)

Production	Semantic rules
$L \rightarrow E \text{ n}$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

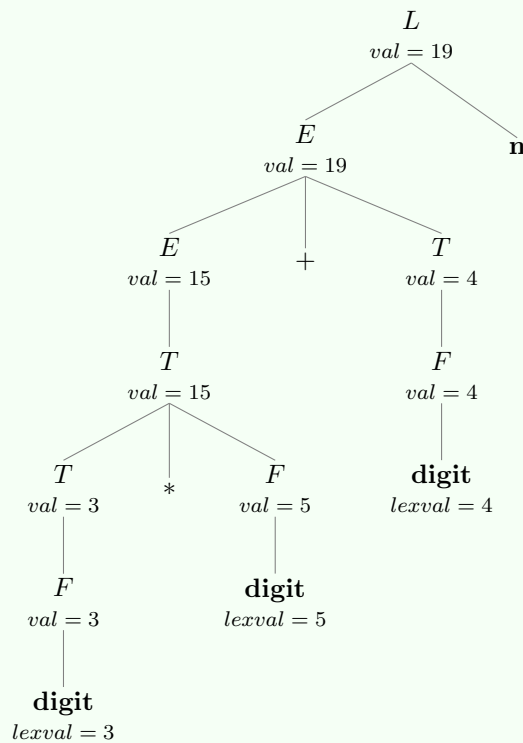


Figure 3: Annotated parse tree for $3*5+4 \text{ n}$. Each node shows the value of its synthesized attribute val .

3.2 Annotated parse trees and dependency graphs

An **annotated parse tree** shows the attribute values at every node. To evaluate attributes we build a **dependency graph**: one node per attribute instance, and an edge $x \rightarrow y$ whenever y is computed from x . A valid evaluation order is any *topological order* of this graph; if the graph has a cycle, no order exists.

Example: inherited attributes (declaration list)

Production	Semantic rules
$D \rightarrow T L$	$L.inh = T.type$
$T \rightarrow \text{int}$	$T.type = integer$
$T \rightarrow \text{float}$	$T.type = float$
$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh; \text{addType}(\text{id.entry}, L.inh)$
$L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$

For the input `float id1, id2` the dashed arrows in Fig. 4 are the dependency-graph edges. The type is passed *down and across* from T to L , then down through the list to each identifier.

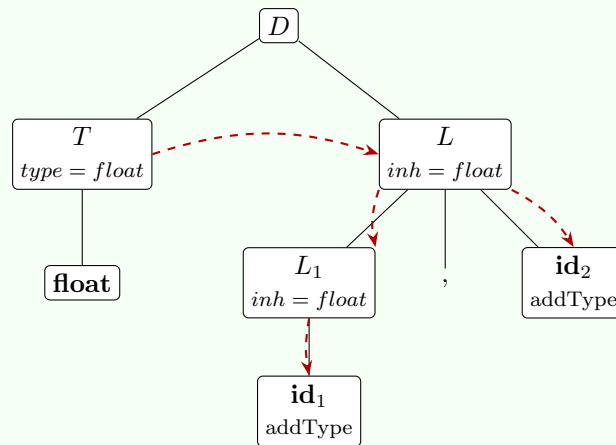


Figure 4: Parse tree for `float id1, id2` with dependency edges (dashed) for the inherited attribute inh .

3.3 Classes of SDDs that are easy to implement

S-attributed and L-attributed definitions

- An SDD is **S-attributed** if *every attribute is synthesized*. It can be evaluated in any bottom-up order, e.g., during LR parsing, at the time of reduction.
- An SDD is **L-attributed** if each attribute is either synthesized, or inherited such that, for a production $A \rightarrow X_1 X_2 \cdots X_n$, the inherited attribute of X_i depends only on:
 - (a) inherited attributes of A , and
 - (b) attributes (inherited or synthesized) of $X_1, \dots, X_{i-1}$ (the symbols to its *left*), and
 - (c) attributes of X_i itself, provided there is no cycle.

Evaluation is possible in a single left-to-right depth-first traversal (e.g., during top-down/LL parsing).

Every S-attributed SDD is also L-attributed.

Not L-attributed

For $A \rightarrow B C$, the rule $C.i = B.s$ is fine (the inherited attribute of C uses its *left* sibling B). But the rule $B.i = f(C.x)$ makes the inherited attribute of B depend on its *right* sibling C , so the SDD is *not* L-attributed.

3.4 SDD for constructing syntax trees

Syntax trees can be built using synthesized attribute *node*, which points to the root of a (sub)tree, and constructor functions $Node(op, c_1, \dots)$ and $Leaf(\mathbf{id}, entry) / Leaf(\mathbf{num}, val)$.

Production	Semantic rules
$E \rightarrow E_1 + T$	$E.node = \mathbf{new} \text{ Node}('+', E_1.node, T.node)$
$E \rightarrow E_1 - T$	$E.node = \mathbf{new} \text{ Node}('-', E_1.node, T.node)$
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow (E)$	$T.node = E.node$
$T \rightarrow \mathbf{id}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, \mathbf{id}.entry)$
$T \rightarrow \mathbf{num}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{num}, \mathbf{num}.val)$

This is S-attributed. If the grammar is made LL by eliminating left recursion, the same tree is built with an inherited attribute (see the L-attributed SDT in Section 4.4).

4 Syntax-Directed Translation Schemes (SDT)**Syntax-Directed Translation Scheme**

A **syntax-directed translation scheme** (SDT) is a context-free grammar with **program fragments** (*semantic actions*) embedded within the production bodies, written in braces. The position of an action defines *when* it is executed during parsing.

$$B \rightarrow X \{a\} Y$$

Here action a is executed after X has been recognised (and before Y).

SDD $\Rightarrow$ *what* to compute (specification, no order); SDT $\Rightarrow$ *how/when* (implementation, order is explicit).

4.1 Postfix translation schemes

If the grammar is LR-parsable and the SDD is **S-attributed**, we can build an SDT with every action at the *right end* of its production (a **postfix SDT**). The actions then execute at *reduction time*, so the attribute of each grammar symbol can be stored beside it on the parser stack.

Postfix SDT for the desk calculator

$L \rightarrow E \mathbf{n}$	$\{ \text{print}(E.val) \}$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow (E)$	$\{ F.val = E.val \}$
$F \rightarrow \mathbf{digit}$	$\{ F.val = \mathbf{digit}.lexval \}$

Implementation on the LR parse stack. Each stack entry holds a grammar symbol and its attribute value. When $A \rightarrow XYZ$ is reduced, the actions read the values in the top three entries ($\text{stack}[\text{top}-2].val$, $\text{stack}[\text{top}-1].val$, $\text{stack}[\text{top}].val$), compute $A.val$, pop three entries, and push A with the new value.

Stack (symbol:val)	Remaining input	Action
\$	3 * 5 + 4 n	shift
\$ digit :3	* 5 + 4 n	reduce $F \rightarrow \mathbf{digit}$
\$ F :3	* 5 + 4 n	reduce $T \rightarrow F$
\$ T :3	* 5 + 4 n	shift *, shift digit
\$ T :3 * digit :5	+ 4 n	reduce $F \rightarrow \mathbf{digit}$
\$ T :3 * F :5	+ 4 n	reduce $T \rightarrow T * F$ ($3 \times 5 = 15$)
\$ T :15	+ 4 n	reduce $E \rightarrow T$
\$ E :15	+ 4 n	shift +, shift digit , reduce to T :4
\$ E :15 + T :4	n	reduce $E \rightarrow E + T$ ($15 + 4 = 19$)
\$ E :19	n	shift n , reduce $L \rightarrow E \mathbf{n}$, print 19

4.2 SDTs with actions inside productions

When an action is in the middle of a production, it is performed *immediately after all grammar symbols to its left have been matched*. In a top-down parser the action is executed at that point of the recursive-descent procedure; in a bottom-up parser it is performed right after the symbols to its left are on the stack.

Example: infix to postfix (action placed after the operand)

$$\begin{aligned}
E &\rightarrow T R \\
R &\rightarrow + T \{ \text{print}(' + ') \} R_1 \mid - T \{ \text{print}(' - ') \} R_1 \mid \varepsilon \\
T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \}
\end{aligned}$$

For 9-5+2 the output is 95-2+.

4.3 Eliminating left recursion from SDTs and marker nonterminals

Marker nonterminals (for bottom-up parsing). Suppose an action must appear in the middle of a production, e.g., $B \rightarrow X \{a\} Y$. For an LR parser we replace the action by a **marker nonterminal** M with an empty production:

$$B \rightarrow X M Y, \quad M \rightarrow \varepsilon \{a\}$$

The action a is then executed upon reducing $M \rightarrow \varepsilon$. (This can introduce conflicts if the grammar is not careful; the marker must not cause a parsing-action conflict.) We will use markers M and N extensively in backpatching (Section 8).

4.4 Converting an L-attributed SDD into an SDT

Rules for converting an L-attributed SDD to an SDT

1. **Inherited attribute** of a nonterminal X in the body: embed the action that computes it *immediately before* X in the body. If several inherited attributes of X depend on one another, order the actions appropriately.
2. **Synthesized attribute** of the head A : place the action that computes it at the *end* of the production body.

Example: syntax tree for a left-recursion-free grammar

After removing left recursion from $E \rightarrow E_1 + T \mid E_1 - T \mid T$:

$$\begin{aligned}
E &\rightarrow T \{ R.i = T.node \} R \{ E.node = R.s \} \\
R &\rightarrow + T \{ R_1.i = \text{new Node}(' + ', R.i, T.node) \} R_1 \{ R.s = R_1.s \} \\
R &\rightarrow - T \{ R_1.i = \text{new Node}(' - ', R.i, T.node) \} R_1 \{ R.s = R_1.s \} \\
R &\rightarrow \varepsilon \{ R.s = R.i \} \\
T &\rightarrow \text{id} \{ T.node = \text{new Leaf}(\text{id}, \text{id.entry}) \}
\end{aligned}$$

Here $R.i$ (inherited) carries the tree built *so far*; $R.s$ (synthesized) returns the final tree. For $a - 4 + c$ the tree $((a - 4) + c)$ is produced with the correct left-associativity.

5 Translation of Expressions and Assignments

5.1 SDD for three-address code

We use the following:

- $E.addr$: the address (name, constant, or temporary) that holds the value of E .
- $E.code$ (or $S.code$): the three-address instructions that evaluate E (or execute S).
- $newTemp()$ (written **new** $Temp$): returns a fresh temporary $t_1, t_2, \dots$ in successive calls.
- $gen(\dots)$: builds a three-address instruction. $\parallel$ denotes concatenation of instruction sequences.

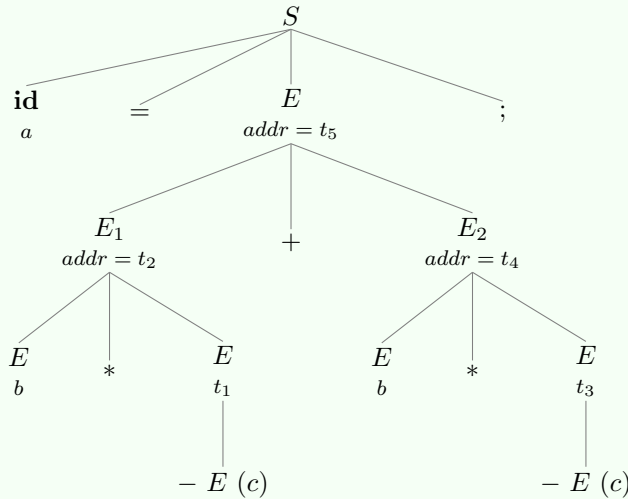
Production	Semantic rules
$S \rightarrow id = E ;$	$S.code = E.code \parallel gen(top.get(id.lexeme) = E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = newTemp()$ $E.code = E_1.code \parallel E_2.code \parallel gen(E.addr = E_1.addr + E_2.addr)$
$E \rightarrow - E_1$	$E.addr = newTemp()$ $E.code = E_1.code \parallel gen(E.addr = minus E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr; \quad E.code = E_1.code$
$E \rightarrow id$	$E.addr = top.get(id.lexeme); \quad E.code = ''$

Table 3: SDD to produce three-address code for expressions and assignments (Dragon Book Fig. 6.19).

Remark

Here top is the current symbol table; $top.get(name)$ returns the symbol-table entry. The code of a node is the *concatenation* of its children's code followed by one new instruction, so the instructions are produced in postfix (evaluation) order. The $.code$ attribute is a long string, so this SDD is inefficient in practice, which motivates the incremental approach below.

Worked example: $a = b * -c + b * -c ;$



Concatenating the code in the order of evaluation gives:

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5

```

5.2 Incremental translation (SDT with gen emitting directly)

Building $E.code$ as a string copies code repeatedly. In an *incremental* scheme, $gen()$ **appends** the instruction to an output stream/array as soon as the action is executed. The code attribute is not stored. Since actions run in the correct order (postfix), the instructions come out in the right order.

Production	Translation scheme (actions in braces)
$S \rightarrow \mathbf{id} = E ;$	$\{ gen(top.get(\mathbf{id.lexeme}) = E.addr); \}$
$E \rightarrow E_1 + E_2$	$\{ E.addr = \mathbf{new Temp}(); gen(E.addr = E_1.addr + E_2.addr); \}$
$\mid -E_1$	$\{ E.addr = \mathbf{new Temp}(); gen(E.addr = \mathbf{minus} E_1.addr); \}$
$\mid (E_1)$	$\{ E.addr = E_1.addr; \}$
$\mid \mathbf{id}$	$\{ E.addr = top.get(\mathbf{id.lexeme}); \}$

Table 4: Incremental SDT for expressions (all actions are at the right end, so it is a postfix SDT, suitable for LR parsing).

5.3 Addressing array elements

Let an array be stored in a contiguous block of memory with base address $base$ and element width w .

One-dimensional array $A[low \dots high]$:

$$\text{address of } A[i] = base + (i - low) \times w$$

If $low = 0$ this is simply $base + i \times w$.

Two-dimensional array (row-major) $A[n_1][n_2]$ with element width w :

$$\text{address of } A[i_1][i_2] = base + (i_1 \times n_2 + i_2) \times w$$

k dimensions $A[n_1] \dots [n_k]$ (row-major, 0-based):

$$base + ((\dots((i_1 n_2 + i_2) n_3 + i_3) \dots) n_k + i_k) \times w$$

Equivalently, using the *widths of sub-arrays* $w_1 = n_2 \dots n_k \cdot w$, $w_2 = n_3 \dots n_k \cdot w, \dots, w_k = w$:

$$base + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k.$$

This form is the one used in the translation scheme.

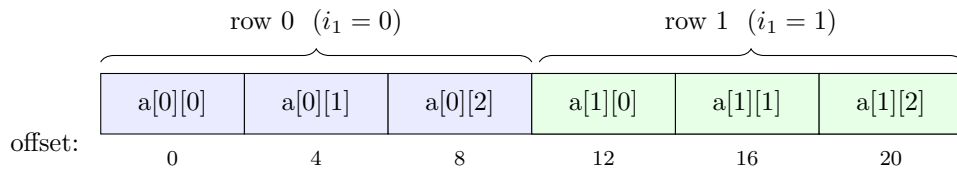


Figure 5: Row-major layout of `int a[2][3]` ($w = 4$). Offset of $a[i][j]$ is $i \times 12 + j \times 4$.

5.4 Translation of array references

Grammar (with nonterminal L for an array location):

$$\begin{aligned} S &\rightarrow \mathbf{id} = E ; \mid L = E ; \\ E &\rightarrow E_1 + E_2 \mid \mathbf{id} \mid L \\ L &\rightarrow \mathbf{id}[E] \mid L_1[E] \end{aligned}$$

Attributes of L : $L.array$ (symbol-table entry of the array), $L.type$ (type of the *sub-array* generated by L), $L.addr$ (a temporary accumulating the *offset*).

Production	Semantic action
$S \rightarrow \mathbf{id} = E ;$	$\{ \text{gen}(\text{top.get}(\mathbf{id.lexeme}) = E.addr); \}$
$S \rightarrow L = E ;$	$\{ \text{gen}(L.array.base [L.addr] = E.addr); \}$
$E \rightarrow E_1 + E_2$	$\{ E.addr = \mathbf{new Temp}(); \text{gen}(E.addr = E_1.addr + E_2.addr); \}$
$E \rightarrow \mathbf{id}$	$\{ E.addr = \text{top.get}(\mathbf{id.lexeme}); \}$
$E \rightarrow L$	$\{ E.addr = \mathbf{new Temp}(); \text{gen}(E.addr = L.array.base [L.addr]); \}$
$L \rightarrow \mathbf{id}[E]$	$\{ L.array = \text{top.get}(\mathbf{id.lexeme}); L.type = L.array.type.elem;$ $L.addr = \mathbf{new Temp}(); \text{gen}(L.addr = E.addr * L.type.width); \}$
$L \rightarrow L_1[E]$	$\{ L.array = L_1.array; L.type = L_1.type.elem;$ $t = \mathbf{new Temp}(); L.addr = \mathbf{new Temp}();$ $\text{gen}(t = E.addr * L.type.width); \text{gen}(L.addr = L_1.addr + t); \}$

Table 5: Semantic actions for array references (Dragon Book Fig. 6.22).

Worked example: $c + a[i][j]$ where a is `int a[2][3]`

Types: $a.type = \text{array}(2, \text{array}(3, \text{integer}))$, so the width of one row is $3 \times 4 = 12$ and of an element is 4.

1. $L \rightarrow \mathbf{id}[E]$ with $E = i$: $L.type = \text{array}(3, \text{integer})$, width 12 $\Rightarrow t1 = i * 12$.
2. $L \rightarrow L_1[E]$ with $E = j$: $L.type = \text{integer}$, width 4 $\Rightarrow t2 = j * 4, t3 = t1 + t2$.
3. $E \rightarrow L$: $t4 = a[t3]$.
4. $E \rightarrow E_1 + E_2$: $t5 = c + t4$.

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = c + t4
```

And for the assignment $x = a[i][j]$ the code is the first four instructions followed by $x = t4$.

6 Type Checking and Type Conversion

Type expressions represent the type of a language construct: basic types (`int`, `float`, `char`, `boolean`, `void`), $\text{array}(n, T)$, *record*, pointer, and function types $s \rightarrow t$.

Type checking comes in two forms: **synthesis** (the type of an expression is built from the types of its sub-expressions, e.g., for $E_1 + E_2$) and **inference** (the type of a construct is determined from the way it is used).

6.1 Type conversion (coercion)

For $E \rightarrow E_1 + E_2$ where operand types may differ (e.g., `int` and `float`), the compiler inserts conversions. Define the widening hierarchy `int < float`. Then

$$E.type = \max(E_1.type, E_2.type),$$

and the helper $widen(a, t, w)$ converts address a of type t to type w :

```
Addr widen(Addr a, Type t, Type w) {
    if (t == w) return a;
    else if (t == integer && w == float) {
        temp = new Temp();
        gen(temp, '(float)', a);
        return temp;
    }
    else error;
}
```

Production	Semantic action
$E \rightarrow E_1 + E_2$	$\{ E.type = \max(E_1.type, E_2.type);$ $a_1 = widen(E_1.addr, E_1.type, E.type);$ $a_2 = widen(E_2.addr, E_2.type, E.type);$ $E.addr = \text{new Temp}(); \text{gen}(E.addr = a_1 + a_2); \}$

Example

Let `x` be `float`, and `i`, `j` be `int`. For $x = i * j + x$ we get:

```
t1 = i * j          // int * int -> int
t2 = (float) t1      // widen to float
t3 = t2 + x          // float + float
x  = t3
```

7 Control Flow: Boolean Expressions and Statements

In programming languages, boolean expressions are used for two purposes:

1. to **compute logical values** (e.g., `flag = a < b && c;`), and
2. as **conditional expressions** in `if`, `while`, and `for` (flow-of-control).

We use the grammar

$$B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$$

with $\parallel$ standing for the C operator `||`; `||` and `&&` are left-associative, `||` has lower precedence than `&&`, which is lower than `!`.

7.1 Short-circuit (jumping) code

In **short-circuit code**, the operators `&&`, `||` and `!` do not appear in the generated code. Instead a boolean expression is translated into jumps to labels: control *reaches* one position if the expression is true and another if it is false. In

```
if (x < 100 || x > 200 && x != y) x = 0;
```

the first test $x < 100$ being true makes the other tests unnecessary. The generated code is:

```

        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:     ...

```

(Compare with *value* code, which would compute $0/1$ into temporaries and use $\&\&$ and $||$ at run time.)

7.2 Flow-of-control statements: SDD with inherited labels

Grammar for statements:

$$P \rightarrow S, \quad S \rightarrow \text{assign} \mid \text{if}(B) S_1 \mid \text{if}(B) S_1 \text{ else } S_2 \mid \text{while}(B) S_1 \mid S_1 S_2$$

Inherited attributes (labels):

- $B.true$: the label to jump to if B is true; $B.false$: the label if B is false.
- $S.next$: the label of the instruction that follows the code for S .

Synthesized attribute: $S.code$, $B.code$. The function $newlabel()$ creates a fresh label $L_1, L_2, \dots$, and $label(L)$ attaches label L to the next instruction.

Production	Semantic rules
$P \rightarrow S$	$S.next = newlabel(); \quad P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if}(B) S_1$	$B.true = newlabel(); \quad B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if}(B) S_1 \text{ else } S_2$	$B.true = newlabel(); \quad B.false = newlabel(); \quad S_1.next = S_2.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel \text{gen}(\text{goto } S.next) \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while}(B) S_1$	$begin = newlabel(); \quad B.true = newlabel(); \quad B.false = S.next;$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code \parallel label(B.true) \parallel S_1.code \parallel \text{gen}(\text{goto } begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel(); \quad S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Table 6: SDD for flow-of-control statements (Dragon Book Fig. 6.36). $S.next$, $B.true$, and $B.false$ are inherited.

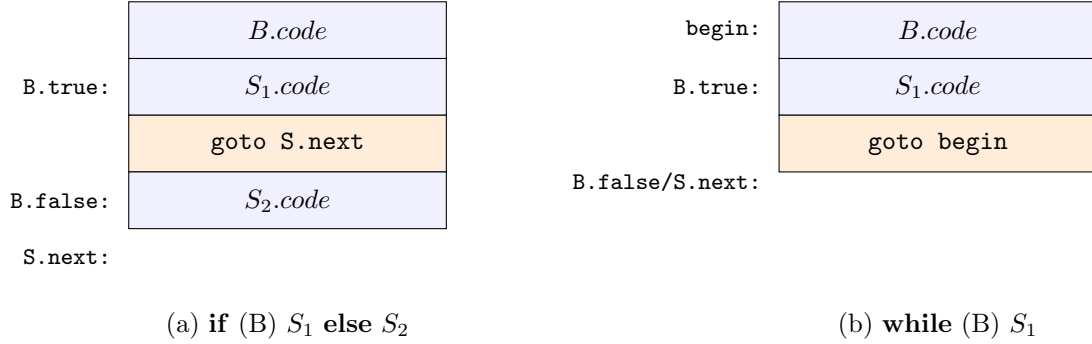


Figure 6: Layout of the generated code. *B.code* contains jumps to *B.true* or *B.false*; it “falls through” to neither, so the labels can be placed anywhere.

Production	Semantic rules
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true; B_1.false = newlabel(); B_2.true = B.true; B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel(); B_1.false = B.false; B_2.true = B.true; B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1.true) \parallel B_2.code$
$B \rightarrow !B_1$	$B_1.true = B.false; B_1.false = B.true; B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code \parallel \text{gen}(\text{if } E_1.addr \text{ rel.op } E_2.addr \text{ goto } B.true) \parallel \text{gen}(\text{goto } B.false)$
$B \rightarrow \text{true}$	$B.code = \text{gen}(\text{goto } B.true)$
$B \rightarrow \text{false}$	$B.code = \text{gen}(\text{goto } B.false)$

Table 7: SDD to generate jumping code for boolean expressions (Dragon Book Fig. 6.37).

7.3 SDD for boolean expressions (jumping code)

How to remember the `||` and `&&` rules

- $B_1 || B_2$: if B_1 is true we are done (go to $B.true$); if false, *test* B_2 , so $B_1.false$ is a new label placed in front of $B_2.code$.
- $B_1 \&\& B_2$: if B_1 is false we are done (go to $B.false$); if true, *test* B_2 , so $B_1.true$ is a new label in front of $B_2.code$.
- $!B_1$: simply *swap* the true and false labels.

Worked example: `if (x < 100 || x > 200 && x != y) x = 0;`

Step 1. $P \rightarrow S$: $S.next = L_1$.

Step 2. $S \rightarrow \text{if}(B)S_1$: $B.true = L_2$, $B.false = L_1$, $S_1.next = L_1$.

Step 3. $B = B_1 || B_2$, where $B_1 \equiv x < 100$ and $B_2 \equiv (x > 200 \&\& x \neq y)$:

- B_1 : true = L_2 , false = L_3 (new).
- B_2 : true = L_2 , false = L_1 .

Step 4. $B_2 = B_3 \&\& B_4$ with $B_3 \equiv x > 200$, $B_4 \equiv x \neq y$:

- B_3 : true = L_4 (new), false = L_1 .
- B_4 : true = L_2 , false = L_1 .

Step 5. Assemble: $B_1.code || L_3: B_2.code$, with $B_2.code = B_3.code || L_4: B_4.code$, followed by $L_2: S_1.code$ and finally L_1 :

```

        if x < 100 goto L2
        goto L3
L3:      if x > 200 goto L4
        goto L1
L4:      if x != y goto L2
        goto L1
L2:      x = 0
L1:
```

Observe the redundant jumps (`goto L3` immediately followed by `L3:`) – a later optimisation phase or the *backpatching* method with fall-through can remove them.

Worked example: `while (a < b) { if (c < d) x = y + z; }`

```

L1:      if a < b goto L2          // begin / B.code
        goto L0                  // B.false = S.next
L2:      if c < d goto L3          // inner if: B.true = L3
        goto L1                  // inner S.next = begin = L1
L3:      t1 = y + z
        x = t1
        goto L1                  // goto begin
L0:                        // S.next
```

7.4 Avoiding redundant gotos (fall-through)

Instead of

```

        if x > 200 goto L4
        goto L1
L4:      ...
```

we can generate the shorter form `ifFalse x > 200 goto L1` followed by the code at `L4`: if the true

label is the next instruction (denote it by the special label *fall*), only one jump is needed. The rule for $B \rightarrow E_1 \text{ rel } E_2$ becomes: let $\text{test} = E_1.\text{addr rel.op } E_2.\text{addr}$:

Condition	Generated jump code
$B.\text{true} \neq \text{fall}$ and $B.\text{false} \neq \text{fall}$	if test goto B.true goto B.false
$B.\text{true} \neq \text{fall}$ and $B.\text{false} = \text{fall}$	if test goto B.true
$B.\text{true} = \text{fall}$ and $B.\text{false} \neq \text{fall}$	ifFalse test goto B.false
both = fall	(no instruction)

8 Backpatching

8.1 The problem and the idea

In the one-pass, label-based scheme above we needed *inherited* labels ($B.\text{true}$, $S.\text{next}$) because a jump may be generated *before* its target is known. Inherited attributes make one-pass **bottom-up** translation awkward. **Backpatching** avoids this:

- When a jump is generated, its *target is left blank*.
- Each such incomplete jump is put on a **list** of jumps that will eventually go to the same target.
- When the target address becomes known, the instructions on the list are *patched* (filled in).

Thus we only need *synthesized* attributes (lists of instruction indices), and the whole translation can be done in a single bottom-up pass.

Backpatching primitives

- $\text{makelist}(i)$: create a new list containing only i , an index into the instruction array; return a pointer to it.
- $\text{merge}(p_1, p_2)$: concatenate the lists pointed to by p_1 and p_2 ; return the merged list.
- $\text{backpatch}(p, i)$: insert i as the target label for each instruction on the list p .
- nextinstr : global variable containing the index of the next instruction to be generated. $\text{emit}(\dots)$ appends an instruction and increments it.

8.2 Boolean expressions

Synthesized attributes: $B.\text{truelist}$ (instructions that jump when B is true) and $B.\text{falselist}$ (when false). A **marker nonterminal** $M \rightarrow \varepsilon$ records the index of the next instruction ($M.\text{instr}$).

Worked example: $x < 100 \parallel x > 200 \ \&\& \ x \neq y$ (instructions start at index 100)

Parse structure: $B \Rightarrow B_1 \parallel M_1 B_2$, where $B_2 \Rightarrow B_3 \ \&\& \ M_2 B_4$.

1. $B_1 \equiv x < 100$: emit 100: if x<100 goto -, 101: goto -
 $B_1.\text{truelist} = \{100\}$, $B_1.\text{falselist} = \{101\}$.
2. $M_1.\text{instr} = 102$.
3. $B_3 \equiv x > 200$: emit 102: if x>200 goto -, 103: goto -
 $B_3.\text{truelist} = \{102\}$, $B_3.\text{falselist} = \{103\}$.
4. $M_2.\text{instr} = 104$.

Production	Semantic action
$B \rightarrow B_1 \parallel M B_2$	$\{ \text{backpatch}(B_1.\text{falselist}, M.\text{instr});$ $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist}); B.\text{falselist} = B_2.\text{falselist}; \}$
$B \rightarrow B_1 \&\& M B_2$	$\{ \text{backpatch}(B_1.\text{truelist}, M.\text{instr});$ $B.\text{truelist} = B_2.\text{truelist}; B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist}); \}$
$B \rightarrow ! B_1$	$\{ B.\text{truelist} = B_1.\text{falselist}; B.\text{falselist} = B_1.\text{truelist}; \}$
$B \rightarrow (B_1)$	$\{ B.\text{truelist} = B_1.\text{truelist}; B.\text{falselist} = B_1.\text{falselist}; \}$
$B \rightarrow E_1 \text{ rel } E_2$	$\{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1);$ $\text{emit}(\text{if } E_1.\text{addr rel.op } E_2.\text{addr goto } -); \text{emit}(\text{goto } -); \}$
$B \rightarrow \text{true}$	$\{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } -); \}$
$B \rightarrow \text{false}$	$\{ B.\text{falselist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } -); \}$
$M \rightarrow \varepsilon$	$\{ M.\text{instr} = \text{nextinstr}; \}$

Table 8: Translation scheme for boolean expressions using backpatching (Dragon Book Fig. 6.43).

5. $B_4 \equiv x \neq y$: emit 104: if x!=y goto -, 105: goto -
 $B_4.\text{truelist} = \{104\}, B_4.\text{falselist} = \{105\}.$
6. Reduce $B_2 \rightarrow B_3 \&\& M_2 B_4$: $\text{backpatch}(\{102\}, 104) \Rightarrow$ instruction 102 becomes if x>200 goto 104.
 $B_2.\text{truelist} = \{104\}, B_2.\text{falselist} = \text{merge}(\{103\}, \{105\}) = \{103, 105\}.$
7. Reduce $B \rightarrow B_1 \parallel M_1 B_2$: $\text{backpatch}(\{101\}, 102) \Rightarrow$ instruction 101 becomes goto 102.
 $B.\text{truelist} = \text{merge}(\{100\}, \{104\}) = \{100, 104\}, B.\text{falselist} = \{103, 105\}.$

Result (still incomplete – the lists will be patched by the enclosing statement):

```

100:  if x < 100 goto _      <- on truelist
101:  goto 102
102:  if x > 200 goto 104
103:  goto _                <- on falselist
104:  if x != y goto _      <- on truelist
105:  goto _                <- on falselist

```

8.3 Flow-of-control statements

Synthesized attribute $S.\text{nextlist}$: list of jumps to the instruction that follows S . We use markers $M \rightarrow \varepsilon$ (records nextinstr) and $N \rightarrow \varepsilon$ (emits an incomplete **goto** and returns it on a list).

Grammar:

$$\begin{aligned}
 S &\rightarrow \text{if } (B) S_1 \mid \text{if } (B) S_1 \text{ else } S_2 \mid \text{while } (B) S_1 \mid \{ L \} \mid A; \\
 L &\rightarrow L_1 S \mid S
 \end{aligned}$$

Why the markers?

M remembers a position: “this is where the code for S_1 (or for the test) begins”. N generates the jump over the else-part and returns it for later patching. In the **while** rule the loop-back jump uses M_1 , which was recorded before $B.\text{code}$ was generated.

Production	Semantic action
$S \rightarrow \text{if } (B) \ M \ S_1$	$\{ \text{backpatch}(B.\text{truelist}, \ M.\text{instr}); \ S.\text{nextlist} = \text{merge}(B.\text{falselist}, \ S_1.\text{nextlist}); \}$
$S \rightarrow \text{if } (B) \ M_1 \ S_1 \ N \ \text{else } M_2 \ S_2$	$\{ \text{backpatch}(B.\text{truelist}, \ M_1.\text{instr}); \text{backpatch}(B.\text{falselist}, \ M_2.\text{instr});$ $\text{temp} = \text{merge}(S_1.\text{nextlist}, \ N.\text{nextlist}); \ S.\text{nextlist} = \text{merge}(\text{temp}, \ S_2.\text{nextlist}); \}$
$S \rightarrow \text{while } M_1(B) \ M_2 \ S_1$	$\{ \text{backpatch}(S_1.\text{nextlist}, \ M_1.\text{instr}); \text{backpatch}(B.\text{truelist}, \ M_2.\text{instr});$ $S.\text{nextlist} = B.\text{falselist}; \text{emit}(\text{goto } M_1.\text{instr}); \}$
$S \rightarrow \{ L \}$	$\{ S.\text{nextlist} = L.\text{nextlist}; \}$
$S \rightarrow A;$	$\{ S.\text{nextlist} = \text{null}; \}$
$L \rightarrow L_1 \ M \ S$	$\{ \text{backpatch}(L_1.\text{nextlist}, \ M.\text{instr}); \ L.\text{nextlist} = S.\text{nextlist}; \}$
$L \rightarrow S$	$\{ L.\text{nextlist} = S.\text{nextlist}; \}$
$M \rightarrow \varepsilon$	$\{ M.\text{instr} = \text{nextinstr}; \}$
$N \rightarrow \varepsilon$	$\{ N.\text{nextlist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } -); \}$

Table 9: Translation of statements with backpatching (Dragon Book Fig. 6.46).

Worked example: while (a < b) { if (c < d) x = y + z; }

Assume numbering starts at 100; $M_1.\text{instr} = 100$.

1. $B \equiv a < b$: 100: if a<b goto -, 101: goto -; $B.\text{truelist} = \{100\}$, $B.\text{falselist} = \{101\}$.
2. $M_2.\text{instr} = 102$.
3. Inner $B' \equiv c < d$: 102: if c<d goto -, 103: goto -; $B'.\text{truelist} = \{102\}$, $B'.\text{falselist} = \{103\}$.
4. $M.\text{instr} = 104$; assignment $x = y + z$: 104: t1 = y + z, 105: x = t1; $S_1.\text{nextlist} = \text{null}$.
5. Inner if: $\text{backpatch}(\{102\}, 104)$; $S.\text{nextlist} = \text{merge}(\{103\}, \text{null}) = \{103\}$.
6. while: $\text{backpatch}(\{103\}, 100)$ (i.e., 103: goto 100); $\text{backpatch}(\{100\}, 102)$ (i.e., 100: if a<b goto 102); $S.\text{nextlist} = B.\text{falselist} = \{101\}$; emit 106: goto 100.

```

100: if a < b goto 102
101: goto _           <- S.nextlist, patched by whoever follows
102: if c < d goto 104
103: goto 100
104: t1 = y + z
105: x = t1
106: goto 100
107: ...             <- (101 will be backpatched with 107)

```

Worked example: if (a < b) x = 1; else x = 2;

```

100: if a < b goto 102    // B.truelist = {100}, patched to M1 = 102
101: goto 104            // B.falselist = {101}, patched to M2 = 104
102: x = 1
103: goto _             // N: on S.nextlist
104: x = 2
105: ...                // 103 is patched to 105 by the enclosing construct

```

8.4 Comparison: inherited labels vs. backpatching

	Label-based (Fig. 6.36/6.37)	SDD Backpatching (Fig. 6.43/6.46)
Attributes	Inherited labels (<i>true</i> , <i>false</i> , <i>next</i>)	Synthesized lists (<i>truelist</i> , <i>falselist</i> , <i>nextlist</i>)
Class of SDD	L-attributed	S-attributed (with markers <i>M</i> , <i>N</i>)
Best suited for	Top-down / recursive-descent (or AST traversal)	Bottom-up (LR) parsing, single pass
Code storage	Attribute <i>.code</i> , or incremental <i>gen</i>	Instruction array; patched later
Labels	Symbolic labels generated eagerly	Instruction indices known late

9 Switch Statements and Procedure Calls (Brief)

9.1 Switch statement

```
switch (E) {
  case V1: S1
  case V2: S2
  ...
  case V(n-1): S(n-1)
  default: Sn
}
```

Translation scheme: evaluate E into a temporary t , then a sequence of tests, followed by the code of each case.

```
      code to evaluate E into t
      goto test
L1:   code for S1
      goto next
L2:   code for S2
      goto next
...
Ln:   code for Sn
      goto next
test: if t = V1 goto L1
      if t = V2 goto L2
      ...
      if t = V(n-1) goto L(n-1)
      goto Ln
next:
```

Putting the tests *after* the case bodies lets the code generator recognise the test sequence and implement it with a **jump table** (for dense values) or binary search. The **case** instruction can also be used: `case t V1 L1 ... case t Vn Ln; case t t Ldefault.`

9.2 Procedure calls

For a call $id(E_1, E_2, \dots, E_n)$:

```
      code to evaluate E1 into t1
      code to evaluate E2 into t2
      ...
```

```

code to evaluate En into tn
param t1
param t2
...
param tn
call id, n          // or: y = call id, n    if a value is returned

```

Grammar and actions (the parameter addresses are collected in a queue, and then emitted as **param**):

$$\begin{aligned}
 S &\rightarrow \text{call id} (Elist) \quad \{ \text{for each } t \text{ in queue: } gen(\text{param } t); gen(\text{call id.addr}, size(queue)); \} \\
 Elist &\rightarrow Elist, E \quad \{ \text{append } E.addr \text{ to the end of } queue; \} \\
 Elist &\rightarrow E \quad \{ queue = [E.addr]; \}
 \end{aligned}$$

10 A Complete Worked Example

Translating a small program fragment to three-address code

Source fragment (all variables are `int`; `a` is `int a[10]`, so $w = 4$):

```

i = 0;
while (i < n && a[i] != key)
    i = i + 1;

```

Using the label-based SDD. $P \rightarrow S$ gives $S.next = L_0$. For the sequence $S_1 S_2$ (the assignment, then the loop) a label $L_1 = S_1.next$ is placed between them. For the loop, $begin = L_2$, $B.true = L_3$, $B.false = L_0$, and the body's $S_1.next = L_2$. For $B = B_1 \&\& B_2$ a new label $L_4 = B_1.true$ is created, with $B_1.false = L_0$, $B_2.true = L_3$, $B_2.false = L_0$.

```

        i = 0
L1:
L2:      if i < n goto L4      // B1 (begin of while)
        goto L0              // B1 false -> exit loop
L4:      t1 = i * 4           // B2: a[i] != key
        t2 = a[t1]
        if t2 != key goto L3
        goto L0              // B false -> exit loop
L3:      t3 = i + 1           // loop body
        i = t3
        goto L2              // back to begin
L0:

```

(L1 and L2 label the same instruction; a later pass can merge them.)

Using backpatching (instruction numbering from 100). Here $M_1.instr = 101$ (start of loop), the marker of `&&` gives 103, and $M_2.instr = 107$ (start of body).

```

100: i = 0
101: if i < n goto 103          // patched by the && marker
102: goto _                   // on B.falselist = S.nextlist
103: t1 = i * 4
104: t2 = a[t1]
105: if t2 != key goto 107     // truelist patched with M2 = 107
106: goto _                   // on B.falselist = S.nextlist
107: t3 = i + 1
108: i = t3
109: goto 101                  // emitted by the while rule (M1)
110: ...                      // {102, 106} are backpatched with 110

```

Both of the incomplete jumps at 102 and 106 form $S.nextlist$ of the loop; whoever follows the

loop will patch them with the index of the next instruction, here 110.

11 Summary and Key Takeaways

- Intermediate code (syntax trees, DAGs, three-address code in quadruple/triple form) decouples front and back ends and enables machine-independent optimisation.
- An **SDD** specifies semantics via attributes and rules; **synthesized** attributes flow upward, **inherited** attributes flow downward or sideways. **S-attributed** $\subset$ **L-attributed**.
- An **SDT** embeds the actions in the grammar and fixes the order of evaluation. S-attributed SDDs $\rightarrow$ *postfix SDTs* (bottom-up). L-attributed SDDs can be implemented during top-down parsing, or, with marker nonterminals, during bottom-up parsing.
- Expressions: *E.addr* holds the result; *gen* emits TAC; temporaries are created by *newTemp*. Array references use offsets computed from the widths of sub-arrays.
- Boolean expressions are translated into **jumping code**, using the inherited labels *true/false*. Statements use *next*.
- **Backpatching** replaces inherited labels by synthesized lists (*makelist*, *merge*, *backpatch*), allowing one-pass bottom-up generation with markers *M* and *N*.

12 Practice Problems

- Q1.** Translate $x = a * (b + c) - d / (b + c)$ into (a) a syntax tree and DAG, (b) three-address code, (c) quadruples, triples, and indirect triples.
- Q2.** Using the SDD of Table 3, annotate the parse tree and generate TAC for $a = -(b + c) * (b + c)$.
- Q3.** For `int a[5][4]` (int width = 4), write the TAC for $x = a[i][j] + a[j][i]$. Give the width used at each dimension.
- Q4.** Generate jumping code (with inherited labels) for `if (a < b && !(c < d)) x = 1; else x = 2;`
- Q5.** Show, step by step, the *truelist/false list* and the final code (with backpatching, starting at 100) for $a < b \parallel c < d \&\& e < f$.
- Q6.** Using backpatching, translate: `while (x < 10) { x = x + 1; if (x == 5) y = 0; }`
- Q7.** Classify as S-attributed, L-attributed, or neither: (a) $A \rightarrow BC$ with $C.i = B.s$ and $A.s = C.s$; (b) $A \rightarrow BC$ with $B.i = C.s$; (c) the SDD of Table 3.
- Q8.** Write the SDT for $E \rightarrow E_1 * E_2 \mid E_1 + E_2 \mid (E) \mid \text{id}$ that prints the *postfix* form, and show its output for `a + b * (c + d)`.

Answers to selected problems

Q1(b). The DAG shares $b + c$, so the TAC is

```

t1 = b + c
t2 = a * t1
t3 = d / t1
t4 = t2 - t3
x  = t4

```

Q3. Row width = $4 \times 4 = 16$; element width = 4.

```

t1 = i * 16
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = j * 16
t6 = i * 4
t7 = t5 + t6
t8 = a[t7]
t9 = t4 + t8
x  = t9

```

Q4. $B.true = L_2$, $B.false = L_3$, $S.next = L_1$; for B_1 & B_2 with $B_2 = !B_3$: $B_1.true = L_4$, $B_3.true = L_3$, $B_3.false = L_2$.

```

        if a < b goto L4
        goto L3
L4:      if c < d goto L3
        goto L2
L2:      x = 1
        goto L1
L3:      x = 2
L1:

```

Q7. (a) L-attributed ($C.i$ depends on the left sibling B) but not S-attributed. (b) Neither (inherited attribute of B depends on the right sibling C). (c) S-attributed (and hence L-attributed).

Q8. Postfix SDT: $E \rightarrow E_1 * E_2 \{print('*')\}$, $E \rightarrow E_1 + E_2 \{print('+')\}$, $E \rightarrow (E)$, $E \rightarrow id \{print(id.lexeme)\}$ (with precedence/associativity declared or via an unambiguous grammar). Output for $a + b * (c + d)$: $a \ b \ c \ d \ + \ * \ +$.

References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2007. – Chapter 5 (Syntax-Directed Translation) and Chapter 6 (Intermediate-Code Generation).